

Mapas auto-organizativos

Inteligencia Computacional · FICH-UNL · Diego Milone

Diapositivas de *Mapas autoorganizativos* (62 láminas), las dos erratas y las transcripciones 001 a 007

Notación: N es la dimensión de la entrada, M la cantidad de neuronas del mapa, L la cantidad de patrones de entrenamiento y n la iteración (un patrón por iteración). $\mathbf{w}_j \in \mathbb{R}^N$ es el vector de pesos de la neurona j ; G o j^* es la ganadora. En la parte de LVQ la cátedra cambia de nombres: el vector de pesos pasa a llamarse **prototipo** $\mathbf{m}_i$ y la velocidad de aprendizaje η pasa a llamarse α . Es la misma cosa.

Las figuras 1, 2, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14 y 15 reconstruyen contenido que en las diapositivas no está: son las láminas que el profesor desarrollaba en el pizarrón (todo el bloque “Formación de mapas topológicos” son siete viñetas sin una sola figura) o simulaciones hechas para verificar lo que la clase afirma. Todos los números que aparecen en el apunte salen de las corridas de `./imagenes/graficos_som.py`.

1. Auto-organización

Definición (Turing, 1952): la auto-organización es el proceso por el cual, **a partir de interacciones locales, emerge un ordenamiento global**. Nadie coordina desde afuera; cada elemento sigue reglas elementales y la estructura completa termina exhibiendo un comportamiento mucho más complejo que esas reglas.

Los cuatro ejemplos que da la clase, en orden de cercanía al tema:

- **Las hormigas.** No hay un cerebro central que reparta tareas, y sin embargo el hormiguero recolecta alimento con una organización muy compleja. (Este caso vuelve en la tercera parte de la materia, en inteligencia colectiva.)
- **Los alumnos del curso.** Al principio del cuatrimestre se sientan sin estructura; hacia el final hay grupos estables sentados siempre en la misma zona del aula. Nadie asignó los asientos.
- **La corteza cerebral.** Las neuronas relacionadas con el tacto de la mano quedan todas en una misma zona, las del pie en otra, las motoras en otra. Se organizaron así porque **las que recibían información parecida empezaron a funcionar parecido**, y estaban cerca.
- **La retina.** Conos y bastones, bordes y colores. El experimento de los gatos: un gatito criado en una habitación con sólo barras **horizontales** y otro con sólo barras **verticales**; al sacarlos al mundo normal, el primero se chocaba las columnas verticales — su retina literalmente no las veía. **No nacemos sabiendo ver: la retina también se entrena.**

IDEA DE FONDO — de dónde sale la arquitectura

Los dos últimos ejemplos no son adorno: son el origen del modelo. Kohonen (años 80) copió tres cosas del cerebro. (1) Las neuronas están **en un plano**, no en capas apiladas. (2) Cuando una se activa, **influye sobre las que tiene alrededor**. (3) Neuronas cercanas terminan representando **estímulos parecidos**. Toda la red sale de ahí.

Claves de la sección 1

Clave	Qué tenés que poder responder
Definición	Interacciones locales $\rightarrow$ ordenamiento global (Turing, 1952)
Ejemplo canónico	Corteza cerebral / retina: neuronas cercanas, estímulos parecidos
Quién lo propuso	Kohonen, años 80

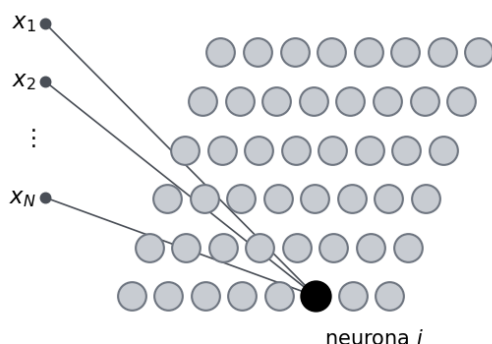
2. Arquitectura del SOM

La arquitectura tiene tres piezas y nada más:

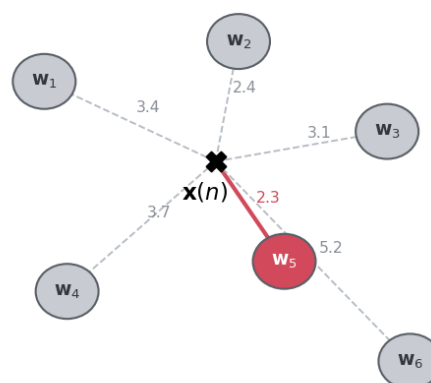
1. Un **vector de entrada** $\mathbf{x} \in \mathbb{R}^N$.
2. Una **matriz de pesos**: cada entrada se conecta con **todas** las neuronas (conexión todos contra todos).

Una sola capa, ordenada en un plano;
cada entrada llega a TODAS las neuronas

$$\mathbf{w}_j = (w_{j1}, w_{j2}, \dots, w_{jN})$$



La salida: gana la neurona cuyo $\mathbf{w}_j$
está más cerca de $\mathbf{x}$



$$j^*(n) = \arg \min_j \|\mathbf{x}(n) - \mathbf{w}_j(n)\|$$

Figura 1: Reconstrucción de las diapositivas 4 a 8. Izquierda: el plano de neuronas y las conexiones. Derecha: la competencia.

3. Una **sola capa** de M neuronas, **ordenadas en un plano** (o en una línea, o en un hexágono).

El vector $\mathbf{w}_j = (w_{j1}, w_{j2}, \dots, w_{jN})^T$ junta **todos los pesos que llegan a la neurona j** , uno por cada entrada. Vive en el mismo espacio $\mathbb{R}^N$ que los datos, y eso es lo que permite dibujarlos juntos en todas las figuras que vienen.

La salida es **competitiva**: ante un patrón se comparan la entrada y **todos** los vectores de peso, y se activa **una sola** neurona, la de menor distancia:

$$j^*(n) = G(\mathbf{x}(n)) = \arg \min_j \{\|\mathbf{x}(n) - \mathbf{w}_j(n)\|\}$$

OJO — no hay función de activación, no hay producto interno

Acá no se calcula $\sum w_{ji}x_i$ ni se le pasa una sigmoide. Se calcula una **distancia** y se toma el mínimo. Es la primera red de la materia donde el peso no es un coeficiente de una combinación lineal sino **un punto del espacio de entrada**: $\mathbf{w}_j$ es la posición de la neurona j dentro de $\mathbb{R}^N$.

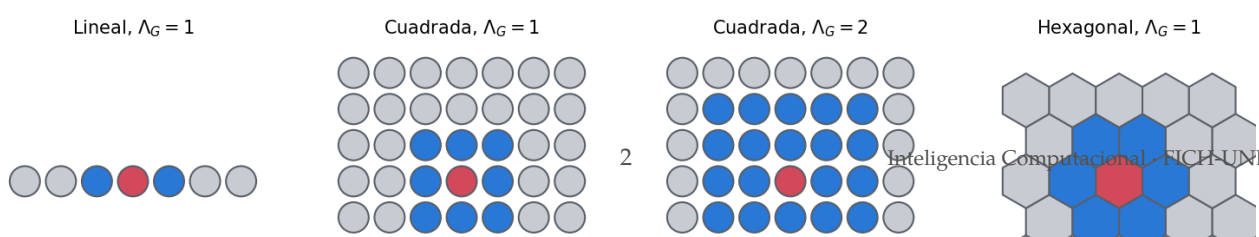
PARA LA DEFENSA — el plano tiene dos espacios y hay que no mezclarlos

Está el **espacio del mapa** (dónde está la neurona respecto de las otras: la cuadrícula, en 1D o 2D) y el **espacio de entrada** $\mathbb{R}^N$ (dónde está su vector de pesos). La vecindad se mide en el **primero**; la distancia que elige a la ganadora, en el **segundo**. Casi todo lo interesante del SOM es la relación entre esos dos espacios, y casi todos los enredos vienen de confundirlos.

Claves de la sección 2

Clave	Qué tenés que poder responder
Capas	Una sola, ordenada en un plano
Conexión	Todos contra todos: cada entrada llega a cada neurona
$\mathbf{w}_j$	Los N pesos que llegan a la neurona j ; es un punto de $\mathbb{R}^N$
Salida	$j^* = \arg \min_j \ \mathbf{x} - \mathbf{w}_j\ $: gana una sola
Los dos espacios	Mapa (vecindad) vs. entrada (distancia)

3. Entornos de influencia o vecindades



- **Alcance Λ_G :** entornos **fijos** o **variables**. En la práctica se usan variables, grandes al principio y decrecientes.
- **Excitación asociada al entorno:** **excitatorias puras**, **inhibitorias puras**, o **funciones generales de excitación/inhibición**.

Claves de la sección 3

Clave	Qué tenés que poder responder
Tres formas	Lineal, cuadrada (la habitual), hexagonal (la original de Kohonen)
Cómo crece Λ_G	Se aplica el criterio recursivamente: radio 2 = las vecinas de las vecinas
Los dos ejes	Alcance (fijo/variable) y tipo de excitación

4. Funciones de excitación/inhibición lateral

$h_{G,i}$ es cuánto le llega de la actualización a la neurona i cuando ganó la G , en función de la distancia $|G - i|$ medida sobre el mapa.

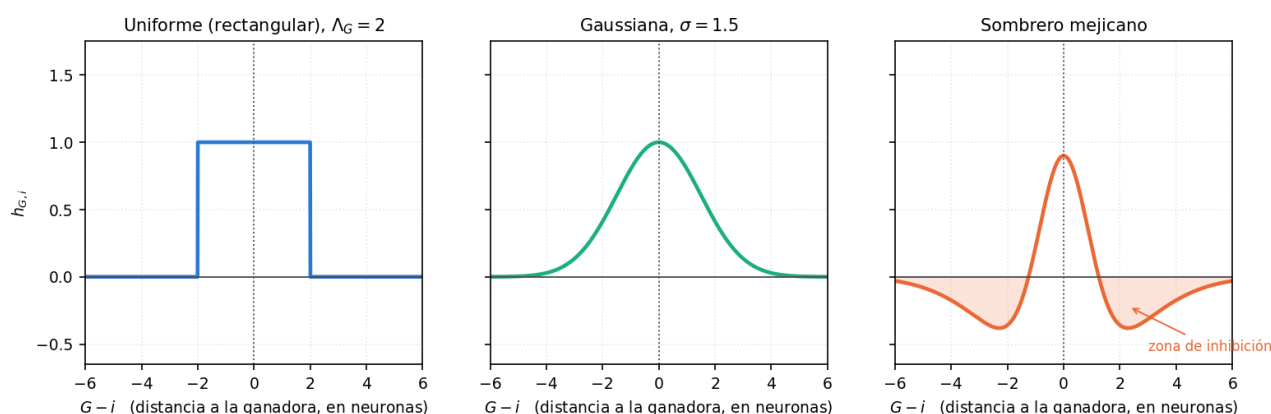


Figura 3: Las tres funciones de la diapositiva 14, dibujadas con escala.

Uniforme (entorno simple o rectangular). Para $\Lambda_G(n) = 2$:

$$\Lambda_G(n) = 2 \Rightarrow \begin{cases} h_{G,i} = \beta(n) & \text{si } |G - i| \leq 2 \\ h_{G,i} = 0 & \text{si } |G - i| > 2 \end{cases}$$

con $\beta(n)$ adaptable en función de las iteraciones n .

Gaussiana.

$$h_{G,i} = \beta(n) e^{-\frac{|G-i|^2}{2\sigma^2(n)}}$$

Acá $\beta(n)$ deja de ser el valor que reciben todas y pasa a ser sólo el **pico**: cuanto más lejos está la neurona de la ganadora, menos la influye. El ancho lo pone $\sigma(n)$, y hacerlo decrecer con n es la forma continua de reducir el entorno.

Sombrero mejicano. Excitación en el centro, **inhibición** en un anillo alrededor, y cero más lejos. Es la que se demostró que funciona en la retina.

IDEA DE FONDO — por qué el sombrero mejicano detecta bordes

Es el experimento que da la clase: mirá fijo un tubo fluorescente o una lámpara, cerrá los ojos, y en la imagen que te queda vas a ver el **borde** del objeto luminoso. La zona iluminada excita a sus neuronas y esas **inhiben a las de alrededor**; el contraste entre lo excitado y lo inhibido cae justo sobre el borde. Ése es el fenómeno de inhibición lateral, y ésa es la razón de que la curva tenga la

panza negativa.

OJO — lo que se usa en la práctica es la aburrida

Las tres funciones entran en el parcial, pero el algoritmo que se implementa usa la **uniforme**: todas las neuronas del entorno se actualizan **igual que la ganadora**, y el resto no se toca. La gaussiana y el sombrero son el marco conceptual (y el nexo con la biología), no lo que se programa.

Claves de la sección 4

Clave	Qué tenés que poder responder
Qué es $h_{G,i}$	Cuánto de la actualización recibe i según su distancia en el mapa a G
Uniforme	Vale $\beta(n)$ dentro del radio, 0 afuera; es la que se implementa
Gaussiana	$\beta(n)$ es el pico; $\sigma(n)$ es el ancho y decrece con n
Sombrero mejicano	Excita en el centro, inhibe alrededor; retina, detección de bordes

5. Entrenamiento: el algoritmo

Dos características que definen todo lo demás:

- Entrenamiento **NO supervisado**: no hay salida deseada en ningún momento. No hay error contra una referencia.
- Aprendizaje **competitivo**: en cada iteración se ajustan **la ganadora y su entorno**, no toda la red.

1. Inicialización. Valores pequeños al azar, $w_{ji} \in [-0,5; +0,5]$ con distribución uniforme; o bien eligiendo al azar $\mathbf{w}_j(0) = \mathbf{x}_\ell$, con $\ell \in [1, \dots, L]$ (un patrón cualquiera del archivo de entrenamiento).

2. Selección del ganador.

$$G(\mathbf{x}(n)) = \arg \min_{\forall j} \{ \|\mathbf{x}(n) - \mathbf{w}_j(n)\| \}$$

3. Adaptación de los pesos.

$$\mathbf{w}_j(n+1) = \begin{cases} \mathbf{w}_j(n) + \eta(n) (\mathbf{x}(n) - \mathbf{w}_j(n)) & \text{si } y_j \in \Lambda_G(n) \\ \mathbf{w}_j(n) & \text{si } y_j \notin \Lambda_G(n) \end{cases}$$

4. Volver a 2 hasta no observar cambios significativos.

Cómo leer la ecuación del paso 3: $\mathbf{x}(n) - \mathbf{w}_j(n)$ es el **vector que va del peso al patrón**; se le suma una fracción η de ese vector, o sea que **el peso se acerca un poco al patrón**. Nada más que eso.

OJO — no es un error de salida, es un vector de posición

En la clase se lo llama “vector de error” y el nombre confunde. En el perceptrón el error era $d - y$, la diferencia contra una salida deseada. Acá no hay salida deseada: $\mathbf{x} - \mathbf{w}_j$ es la **diferencia entre dos puntos del mismo espacio**, y el “aprendizaje” es literalmente mover la neurona hacia el dato. Por eso el entrenamiento es no supervisado aunque la fórmula se parezca a la del descenso por gradiente.

OJO — el entorno se evalúa en el mapa, la distancia en la entrada

$\|\mathbf{x} - \mathbf{w}_j\|$ del paso 2 se mide en $\mathbb{R}^N$. La pertenencia $y_j \in \Lambda_G(n)$ del paso 3 se mide **en la cuadrícula**. Una neurona puede tener el peso lejísimos del patrón y actualizarse igual, sólo porque en el mapa está al lado de la ganadora. **Eso es exactamente lo que produce el ordenamiento topológico.**

Consideraciones prácticas (diapositivas 25 a 28):

- $\Lambda_G(n)$ es generalmente **cuadrado**.
- $h_{G,i}(n)$ **uniforme en i** , decreciente con n .
- $0 < \eta(n) < 1$, **decreciente con n** .

Y de ahí sale la pregunta que la cátedra deja planteada: **¿cómo varían $\Lambda_G(n)$ y $\eta(n)$?** La respuesta es la sección siguiente.

Para la pizarra: el algoritmo de entrenamiento del SOM

Te preguntan: “Escribí el algoritmo de entrenamiento del SOM.”

Arrancás dibujando dos cosas al costado del pizarrón y no las borrás en todo el desarrollo: la cuadrícula del mapa (ahí vive Λ_G) y unos **ejes de $\mathbb{R}^N$** (ahí viven $\mathbf{x}$ y los $\mathbf{w}_j$). Todo lo que escribas después va a pertenecer a uno de los dos dibujos, y decir a cuál es la mitad de la nota.

Paso 1. Inicialización. Decís el rango y la alternativa.

Llegás a: $w_{ji}(0) \sim \mathcal{U}[-0,5; +0,5]$, o bien $\mathbf{w}_j(0) = \mathbf{x}_\ell$ con ℓ sorteado en $[1, \dots, L]$.

Paso 2. Se presenta un patrón $\mathbf{x}(n)$ y compiten **todas** las neuronas. Escribís la competencia y **señalás los ejes de $\mathbb{R}^N$** .

Llegás a: $G(\mathbf{x}(n)) = \arg \min_{\forall j} \{ \|\mathbf{x}(n) - \mathbf{w}_j(n)\| \}$ **La frase:** esto es un mínimo de distancia en el espacio de entrada, no un producto interno ni un máximo de activación.

Paso 3. Definís el entorno de la ganadora y **señalás la cuadrícula**.

Llegás a: $\Lambda_G(n)$, el conjunto de neuronas a distancia de Chebyshev $\leq$ radio **en el mapa**. **La frase:** la pertenencia se decide acá arriba, en la cuadrícula, sin mirar los pesos.

Paso 4. Escribís la adaptación **con las dos ramas**. La segunda rama es la que se olvida y es la que distingue al SOM de k -medias.

Llegás a:

$$\mathbf{w}_j(n+1) = \begin{cases} \mathbf{w}_j(n) + \eta(n) (\mathbf{x}(n) - \mathbf{w}_j(n)) & \text{si } y_j \in \Lambda_G(n) \\ \mathbf{w}_j(n) & \text{si } y_j \notin \Lambda_G(n) \end{cases}$$

Paso 5. Interpretás la primera rama en voz alta señalando los ejes. Esto no es un renglón de más: es lo que demuestra que entendiste que el peso es un punto.

La frase: $\mathbf{x} - \mathbf{w}_j$ es el vector que va del peso al patrón; le sumo una fracción η de ese vector, o sea que muevo la neurona un η del trecho que la separa del dato. No hay error contra una salida deseada: es no supervisado.

Paso 6. Cerrás con el bucle y el criterio de corte, y aclarás que η y Λ_G **no son constantes**.

Llegás a: volver al paso 2 con el patrón siguiente, hasta que no haya cambios significativos en el mapa, con $\eta(n)$ y $\Lambda_G(n)$ **decrecientes con n** — y eso te deja parado justo en la sección 6, que es la continuación natural si te la piden.

Trampa 1: escribir $\arg \max$. Es un **mínimo** de distancia. **Trampa 2:** omitir la segunda rama. Sin ella el desarrollo es k -medias. **Trampa 3:** poner η y Λ_G sin el argumento (n) . Que dependan de n es la sección siguiente entera. **Checkpoint:** si en algún renglón no podés decir **en cuál de los dos dibujos** vive lo que escribiste, algo está mal planteado.

Claves de la sección 5

Clave	Qué tenés que poder responder
Los 4 pasos	Inicializar / elegir ganadora / adaptar entorno / repetir
Inicialización	Al azar en $[-0,5; 0,5]$, o copiando patrones del archivo
La regla	$\mathbf{w}_j += \eta(\mathbf{x} - \mathbf{w}_j)$: acercar el peso al patrón
Quién se actualiza	La ganadora y su entorno ; el resto queda igual
Corte	Cuando no hay cambios significativos en el mapa

6. Las tres etapas del entrenamiento

1. Ordenamiento global (o topológico).

- $\Lambda_G(n)$ **grande**, $\approx$ medio mapa (en un mapa 10×10 , un entorno inicial de 5).
- $\eta(n)$ **grande**, entre 0,9 y 0,7.

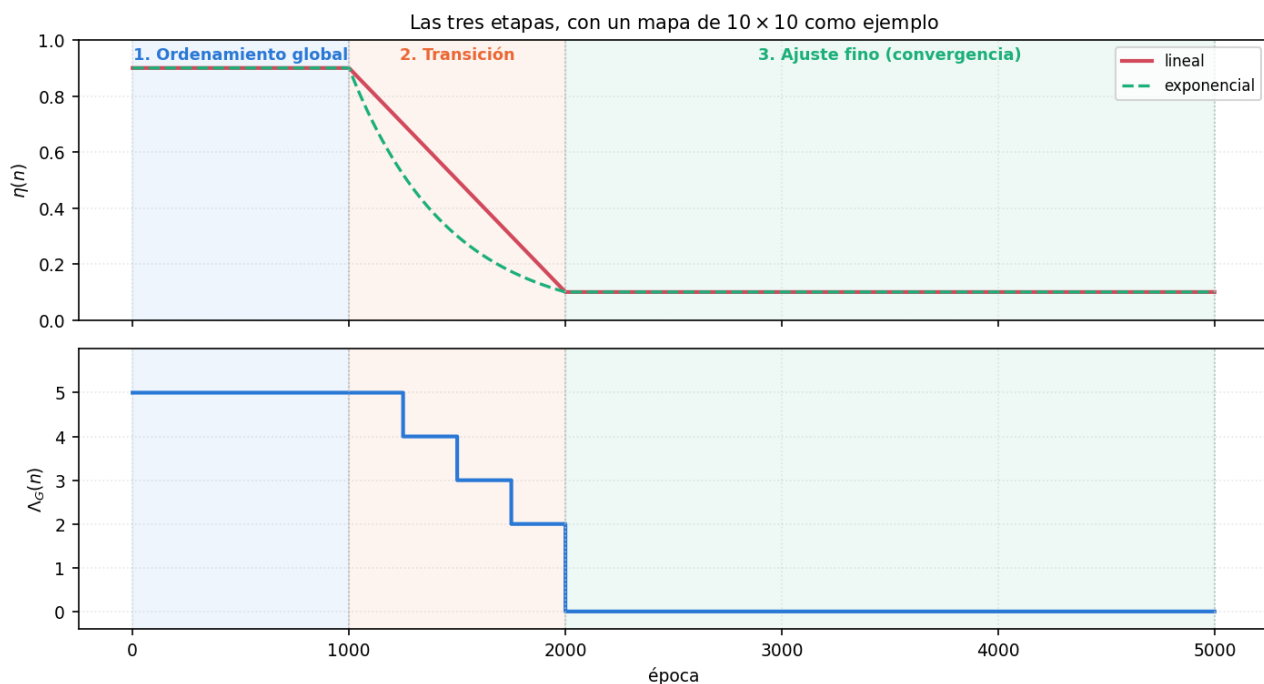


Figura 4: Cómo varían η y Λ_G a lo largo del entrenamiento, con el mapa 10×10 del ejemplo de la clase.

- Duración: 500 a 1000 épocas.

2. Transición.

- $\Lambda_G(n)$ se reduce **linealmente hasta 1**.
- $\eta(n)$ se reduce lineal o exponencialmente **hasta 0,1**.
- Duración: ≈ 1000 épocas.

3. Ajuste fino (o convergencia).

- $\Lambda_G(n) = 0$: **sólo se actualiza la ganadora**.
- $\eta(n) = \eta_{cte}$, entre **0,1 y 0,01**.
- Duración: hasta convergencia, ≈ 3000 épocas.

El ejemplo aritmético que hace la clase para la transición: mapa 10×10 , entorno inicial 5, 1000 épocas de transición. Se usa $\Lambda_G = 5$ en las primeras 200 épocas, 4 entre la 200 y la 400, 3 entre la 400 y la 600, 2 entre la 600 y la 800, y 1 en las últimas 200.

OJO — una época no es una iteración

Una **época** es una pasada completa por el archivo de entrenamiento; una **iteración** n es un solo patrón. Con 1000 patrones, “500 épocas” son 500,000 presentaciones. Los números de arriba están en épocas y son criterios prácticos que funcionaron en muchas aplicaciones, no resultados teóricos.

Verificación: ¿hace falta de verdad la etapa de ordenamiento?

Se entrenaron dos mapas de 8×8 sobre 3000 puntos uniformes en $[-1, 1]^2$, **misma semilla, mismos datos, mismo η** , cambiando una sola cosa: uno arranca con $\Lambda_G = 4$ (etapa de ordenamiento) y el otro con $\Lambda_G = 0$ desde el principio.

Corrida	Cruces de la malla consigo misma
Con etapa de ordenamiento ($\Lambda_G : 4 \rightarrow 0$)	0
Sin etapa de ordenamiento ($\Lambda_G = 0$ siempre)	1328

Las dos corridas cuantizan bien el espacio: las 64 neuronas quedan repartidas sobre el cuadrado en los dos casos. Pero sólo la primera está **ordenada**. Sin la etapa de ordenamiento el mapa aprende los datos y **no aprende la topología**, que es lo único que lo distingue de k -medias.

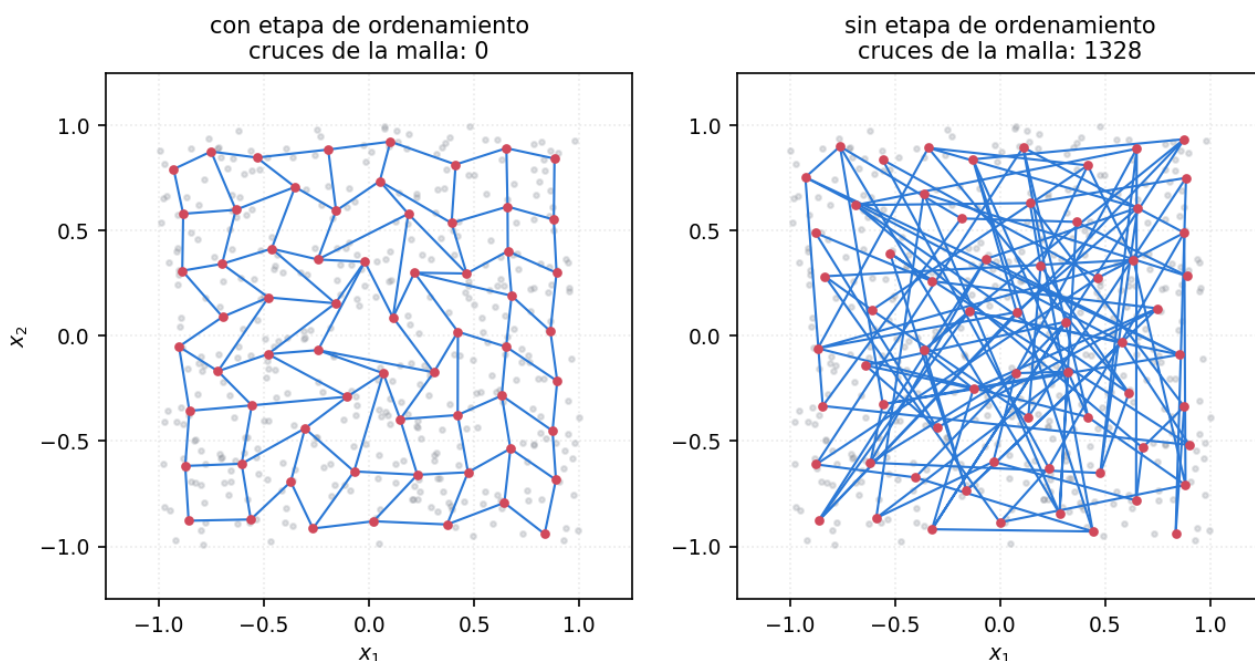


Figura 5: Con y sin etapa de ordenamiento. La malla dibuja las conexiones entre neuronas vecinas **en el mapa**, ubicadas según sus pesos.

PARA LA DEFENSA — el número que conviene tener a mano

“Sin la primera etapa el mapa cuantiza igual de bien pero queda anudado: la malla se cruza a sí misma 1328 veces contra 0.” Es la forma más corta de justificar por qué las tres etapas no son un capricho.

Para la pizarra: armar el cronograma de las tres etapas

Te preguntan: “Tenés un mapa de 10×10 y 2000 épocas de transición. ¿Cómo varían Λ_G y η ?” Es una cuenta de regla de tres, pero hay que ordenarla en voz alta.

Paso 1. Fijás el entorno inicial: **medio mapa**.

Llegás a: mapa de $10 \times 10 \Rightarrow \Lambda_G(0) = 5$.

Paso 2. Contás cuántos valores distintos tiene que tomar Λ_G hasta llegar a 1.

Llegás a: de 5 a 1 son **5 valores** (5, 4, 3, 2, 1).

Paso 3. Repartís las épocas de transición entre esos valores.

Llegás a: $2000/5 = 400$ épocas por valor: $\Lambda_G = 5$ hasta la 400, 4 hasta la 800, 3 hasta la 1200, 2 hasta la 1600 y 1 hasta la 2000. Con las 1000 épocas del apunte el mismo reparto da **200 por valor**, que es el ejemplo que hace la clase.

Paso 4. Hacés lo mismo con η , pero **continuo**: va de su valor al final de la etapa 1 hasta 0,1.

Llegás a: $\eta(n) = \eta_{\text{ini}} - (\eta_{\text{ini}} - 0,1) \frac{n}{N_{\text{transición}}}$ — o exponencial, $\eta(n) = \eta_{\text{ini}} e^{-n/\tau}$; la cátedra acepta las dos.

Paso 5. Cerrás con la etapa 3 y **decís por qué termina así**.

Llegás a: $\Lambda_G = 0$ y η constante entre 0,1 y 0,01. **La frase:** *con el entorno apagado cada neurona ya no arrastra a nadie y termina de caer en el centro de su propio grupo; el orden ya lo consiguió la etapa 1 y no se pierde porque los pasos son chicos.*

Trampa: decir “de 5 a 1 son 4 valores”. Son **5**: hay que contar los dos extremos. **Checkpoint:** las duraciones van en **épocas**, no en iteraciones. Con 1000 patrones, 400 épocas son 400 000 presentaciones.

Claves de la sección 6

Clave	Qué tenés que poder responder
Etapa 1	$\Lambda_G \approx$ medio mapa, $\eta \in [0,7; 0,9]$, 500–1000 épocas
Etapa 2	$\Lambda_G \rightarrow 1$ lineal, $\eta \rightarrow 0,1$, ≈ 1000 épocas
Etapa 3	$\Lambda_G = 0$, $\eta \in [0,01; 0,1]$ cte, ≈ 3000 épocas
Para qué la etapa 1	Para el ordenamiento topológico ; sin ella la malla queda anudada
Para qué la etapa 3	Ajuste fino: cada neurona termina de caer en el centro de su grupo

7. Formación de mapas topológicos

Esta sección entera reconstruye las diapositivas 29 a 36, que son siete viñetas con el enunciado de cada ejemplo y ninguna figura: todo se desarrolló en el pizarrón (transcripciones 004 y 005).

Ejemplo 1 — $\mathbb{R}^1 \rightarrow \mathbb{R}^1$, 2 neuronas

Una sola entrada escalar, dos neuronas, sin entorno. Cada w_j tiene un solo peso, así que **datos y pesos viven en la misma recta** y se pueden dibujar juntos. Los patrones están en dos grupos, alrededor de -1 y de $+1$; los pesos arrancan al azar en $[-0,5; 0,5]$.

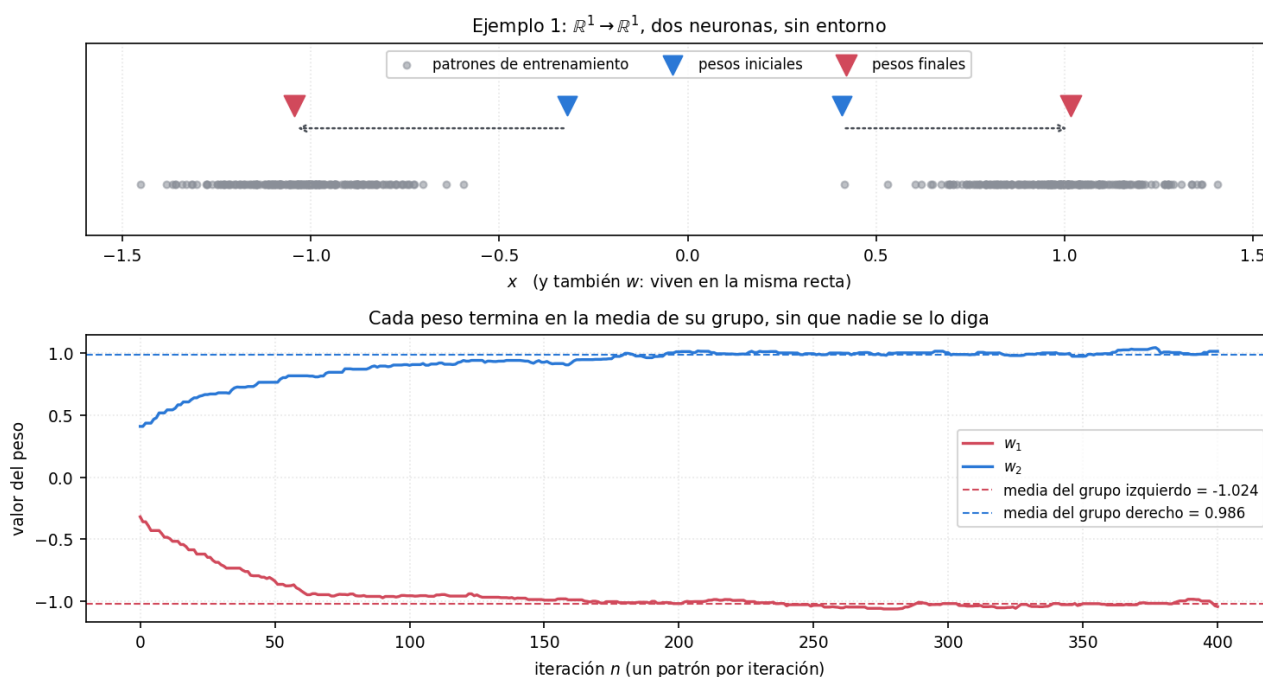


Figura 6: Corrida real: 400 patrones, $\eta = 0,05$, sin entorno.

Cuando entra un patrón del grupo derecho gana el peso que esté más cerca, y ese peso se corre un poco hacia la derecha. Como la mayoría de los patrones de ese grupo le van a tocar a la misma neurona, esa neurona **se va corriendo cada vez más hacia el centro del grupo**, y cada vez gana con más frecuencia. El otro peso queda para el otro grupo.

	w_1	w_2
Valor final tras 400 patrones	-1,045	+1,016
Media real del grupo correspondiente	-1,024	+0,986

Cada peso terminó en la media de su grupo con dos centésimas de diferencia, **sin que nadie le dijera cuántos grupos había ni cuál era cuál**. Eso es el aprendizaje no supervisado, y es lo mismo que hacía k -medias en la unidad de base radial — sólo que en línea, un patrón por vez.

Ejemplo 2 y 3 — $\mathbb{R}^2$, 4 neuronas

Con dos entradas, $\mathbf{w}_j = (w_{j1}, w_{j2})$ es un punto del plano y se dibuja igual. Cuatro grupos de patrones, cuatro neuronas **sin entorno**:

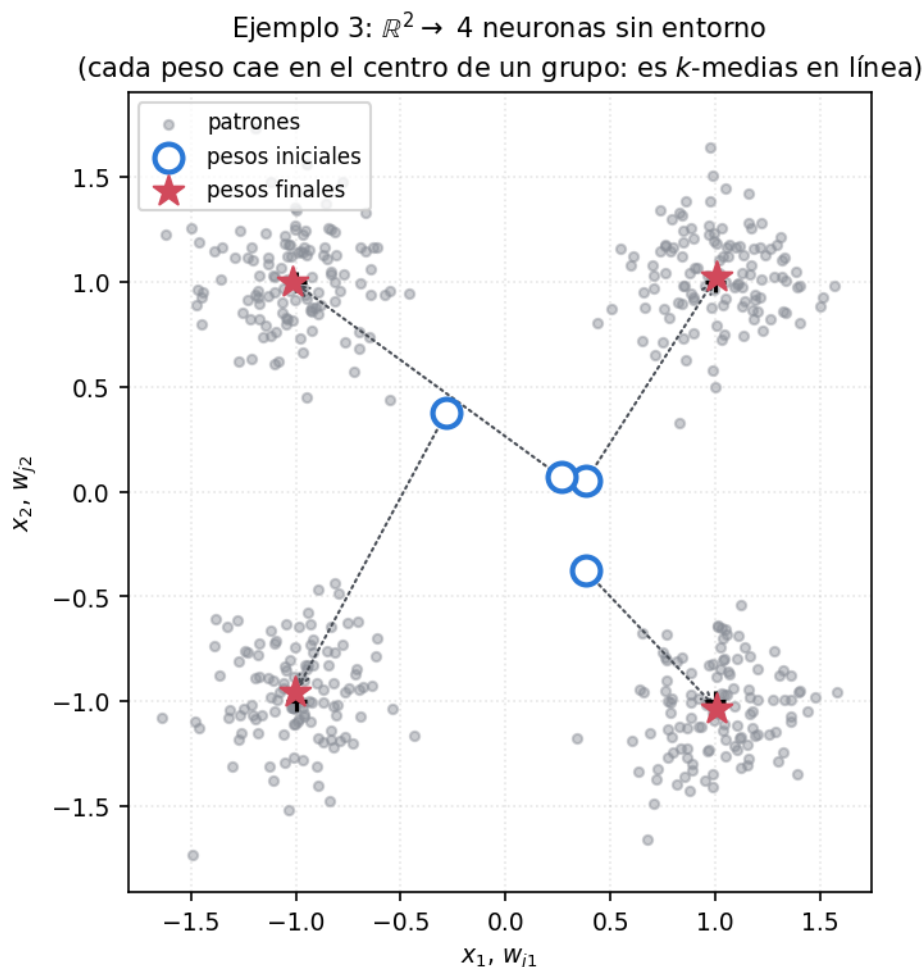


Figura 7: Corrida real: 480 patrones en 4 grupos, 60 épocas, $\eta : 0,30 \rightarrow 0,01$, $\Lambda_G = 0$.

Los cuatro pesos caen en los cuatro centros: el peor de los cuatro quedó a **0,042** del centro de su grupo. Sin entorno, el SOM es k -medias en línea.

OJO — la ganadora no siempre es “la que corresponde”, y no importa

La clase se detiene en esto: en las primeras iteraciones puede ganar una neurona que no era la esperada, porque los pesos arrancaron en cualquier lado. No pasa nada. Lo que decide el resultado es que **para la mayoría de los patrones de un grupo va a ganar la misma neurona**, y esa mayoría la arrastra al centro. El argumento es estadístico, no determinista.

Qué agrega el entorno

Ahora sí con vecindad. El ejemplo de la clase: mapa de 2×2 , neuronas 1 y 2 conectadas entre sí.

Cuando un patrón activa a la 1, como la 1 está conectada con la 2, **también se actualiza la 2**, y al revés. Si los datos que hacen ganar a la 1 están arriba y los que hacen ganar a la 2 también están arriba, las dos van a ser traccionadas hacia arriba **juntas**. Al final, cuando el entorno se apaga, cada una se acomoda en el centro del grupo que le toca — pero ya partiendo de la misma región del espacio.

Para medirlo se entrenaron mapas de 6×6 sobre datos uniformes, 8 inicializaciones distintas, con y sin entorno, y se comparó la distancia media $\|\mathbf{w}_a - \mathbf{w}_b\|$ entre **neuronas vecinas en el mapa** contra la de **un par cualquiera**:

	Vecinas en el mapa	Un par cualquiera
Sin entorno	1,09	1,07

	Vecinas en el mapa	Un par cualquiera
Con entorno	0,38	1,06

Sin entorno, dos neuronas vecinas en la cuadrícula están **tan lejos entre sí como dos neuronas cualesquiera**: la vecindad del mapa no dice absolutamente nada sobre los datos. Con entorno, las vecinas quedan a un tercio de esa distancia. **Ése es el ordenamiento topológico, y es lo único que aporta el entorno.**

Ejemplo 4 — $\mathbb{R}^N \rightarrow \mathbb{R}^2$: el despliegue del pañuelo

El caso general y la razón práctica de todo el método. Una forma de mirar el resultado: **dibujar cada neurona en el lugar que indican sus pesos y unir con una línea a las que son vecinas en el mapa**. Al principio los pesos son pequeños y al azar, así que se ve una maraña apretada cerca del origen; a medida que el mapa se organiza, esa maraña se despliega **como quien abre un pañuelo**, hasta cubrir todo el espacio que tiene que cubrir.

Fijate el segundo cuadro: en la etapa de ordenamiento la malla primero **se contrae** y recién después se estira. Es lo esperable — con $\Lambda_G = 4$ sobre un mapa de 8×8 , casi todas las neuronas se actualizan con casi todos los patrones, así que todos los pesos se van juntos hacia la media de los datos. Sólo cuando el entorno empieza a achicarse cada neurona puede diferenciarse de sus vecinas.

IDEA DE FONDO — para qué sirve todo esto

El interés práctico del SOM no es agrupar (para eso está k -medias). Es que **proyecta un espacio de $\mathbb{R}^N$ que no podés visualizar sobre un mapa de dos dimensiones que sí**, y lo hace conservando la vecindad: patrones parecidos en $\mathbb{R}^{200}$ caen en neuronas cercanas del mapa. Los ejemplos de $\mathbb{R}^2$ se eligen porque se pueden dibujar; el caso que importa es $N = 200$.

Ejemplo 5 — el *phonetic typewriter* (Kohonen)

Mapa **hexagonal**. Se grabó cada fonema del finlandés muchas veces, por distintas personas; de cada grabación sale un vector de análisis en frecuencia, de dimensión alta. Esos vectores se le mostraron al mapa **sin decirle nunca de qué fonema era cada uno**.

Resultado: los fonemas parecidos quedaron agrupados en regiones bien identificadas del mapa — las explosivas por un lado, las vocales por otro. Después Kohonen **etiquetó** cada neurona con el fonema que más veces la había hecho ganar (sección 8). Al pronunciar una palabra, la secuencia de neuronas que se van activando dibuja **un camino sobre el mapa**, y ese camino identifica la palabra.

Ejemplo 6 — el experimento de los países (Kohonen)

Vector de entrada por país: cantidad de habitantes, superficie, renta per cápita, consumo de proteínas, y así. Todos los países en un archivo, **sin decir cuál era cuál**. El mapa los agrupó, y al pintar el mapa del mundo con el color de la neurona que le tocó a cada país aparecen agrupaciones muy reconocibles: casi toda África de un color, Canadá y EEUU del mismo color, Argentina y México del mismo color.

Ejemplo 7 — las demos

Las que trae Matlab, y las direcciones web de la diapositiva 37 (Regensburg, HUT, Bochum, y el buscador gnod.net). Sirven para ver el despliegue del pañuelo animado y con estructuras 3D.

Claves de la sección 7

Clave	Qué tenés que poder responder
Ejemplo 1	2 neuronas en $\mathbb{R}^1$: cada peso termina en la media de su grupo
Sin entorno	El SOM es k -medias en línea
Con entorno	Vecinas en el mapa $\rightarrow$ vectores de peso cercanos: ordenamiento topológico
El pañuelo	La malla arranca apretada, se contrae, y se despliega hasta cubrir el espacio
Para qué sirve	Proyectar $\mathbb{R}^N$ sobre un mapa 2D conservando la vecindad
Phonetic typewriter	Mapa hexagonal de fonemas; cada palabra es un camino sobre el mapa

8. Agrupamiento y clasificación

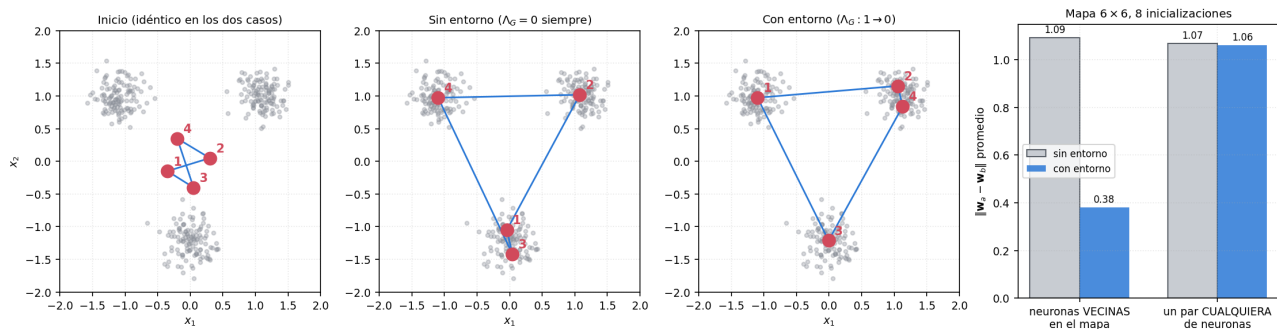


Figura 8: Izquierda y centro: el ejemplo de la clase. Derecha: la verificación sobre un mapa de 6×6 .

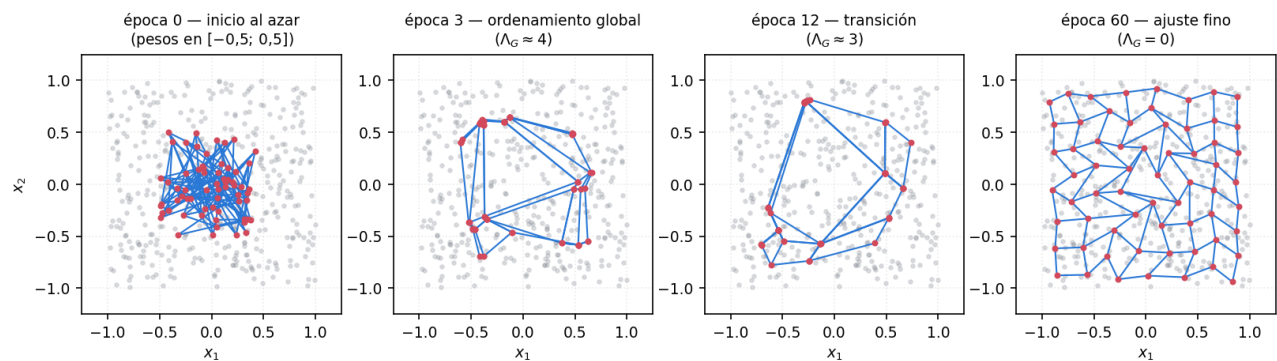


Figura 9: Corrida real: mapa 8×8 , 3000 puntos uniformes en $[-1, 1]^2$, $\eta : 0,60 \rightarrow 0,02$, $\Lambda_G : 4 \rightarrow 0$.

La pregunta de la diapositiva 38: **¿cómo se usa un SOM para clasificar patrones?** Porque hasta acá es un método de agrupamiento no supervisado, como k -medias: nunca le dijimos qué queríamos a la salida.

Tres pasos:

1. **Entrenamiento no supervisado.** El SOM normal, sin usar las clases. (En principio es una desventaja: estamos tirando información que tenemos.)
2. **Etiquetado de neuronas.** Ahora sí se usan las clases. Se le muestran **todos** los patrones y se cuenta, para cada neurona, con cuántos patrones de cada clase ganó. A cada neurona se le pone la etiqueta de la clase **mayoritaria**.
3. **Clasificación por mínima distancia.** Ante un patrón desconocido se busca la ganadora y se responde su **etiqueta**.

En esa corrida el mapa quedó ordenado: las 36 neuronas se etiquetaron todas (ninguna quedó muerta) y las etiquetas forman **tres regiones conexas** en la cuadrícula — no están salpicadas. Sobre los 180 patrones de prueba, que el mapa no vio nunca, la exactitud fue **100 %**.

IDEA DE FONDO — las regiones conexas son la prueba del ordenamiento

Que las A, las B y las C queden en zonas contiguas del mapa **no lo pidió nadie**: el etiquetado se hace neurona por neurona, sin mirar a las vecinas. Sale así porque el ordenamiento topológico ya puso a las neuronas vecinas sobre datos parecidos. Si el mapa saliera con las etiquetas mezcladas, eso sería el síntoma de que la etapa de ordenamiento falló.

OJO — el etiquetado es supervisado, el entrenamiento no

El SOM etiquetado **no** es un método supervisado: la supervisión aparece **después** del entrenamiento, y durante el entrenamiento se perdió la información de clase. Ésa es exactamente la crítica que motiva LVQ (sección 10), que usa las clases **desde la primera iteración**.

Claves de la sección 8

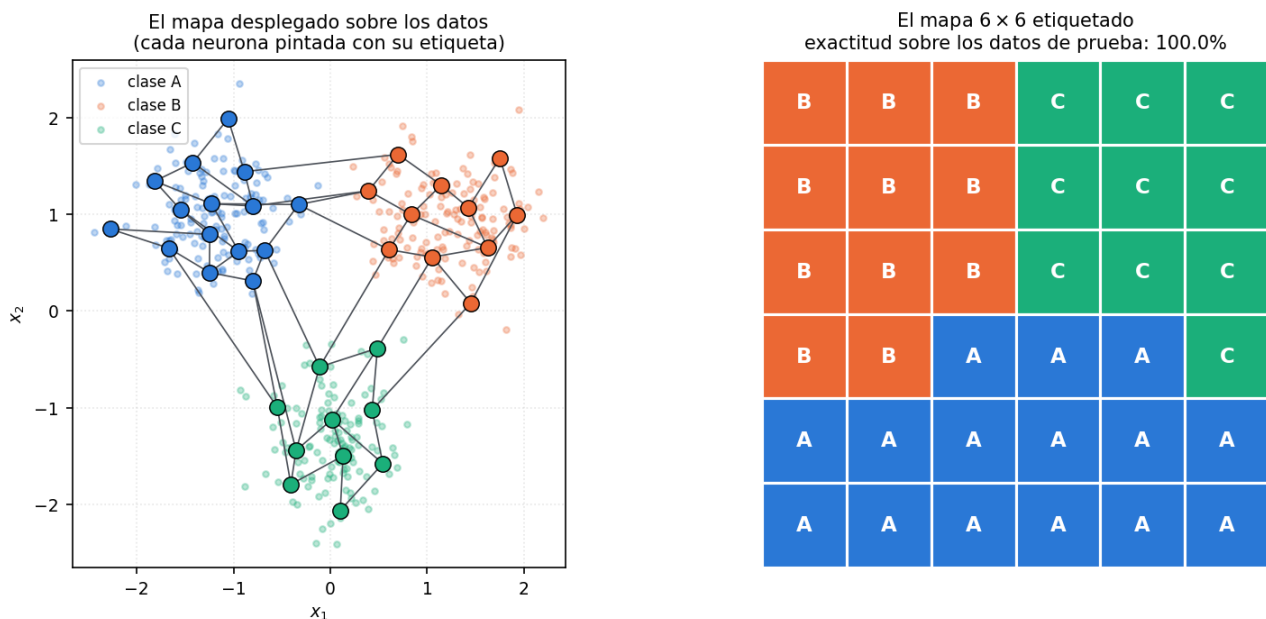


Figura 10: Corrida real: 3 clases gaussianas superpuestas, 420 patrones de entrenamiento y 180 de prueba, mapa 6×6 .

Clave	Qué tenés que poder responder
Los tres pasos	Entrenar no supervisado / etiquetar por mayoría / clasificar por mínima distancia
Cuándo entran las clases	Sólo en el paso 2, después del entrenamiento
Neurona muerta	La que nunca gana: se queda sin etiqueta
Señal de buen mapa	Las etiquetas forman regiones conexas sobre la cuadrícula

9. Cuantización vectorial: los conceptos

Cuantizar es partir un rango continuo en una cantidad finita de valores (*cuantos*), porque en una computadora no hay resolución infinita: hay 8, 16 o 32 bits.

- **Cuantizador escalar:** se cuantiza un eje. Una señal muestreada y llevada a niveles discretos.
- **Cuantizador vectorial:** se cuantiza un **espacio**. El ejemplo más claro es una foto: cada región del plano queda representada por un píxel. Cuando una imagen “está pixelada” es porque los cuantos son demasiado grandes.

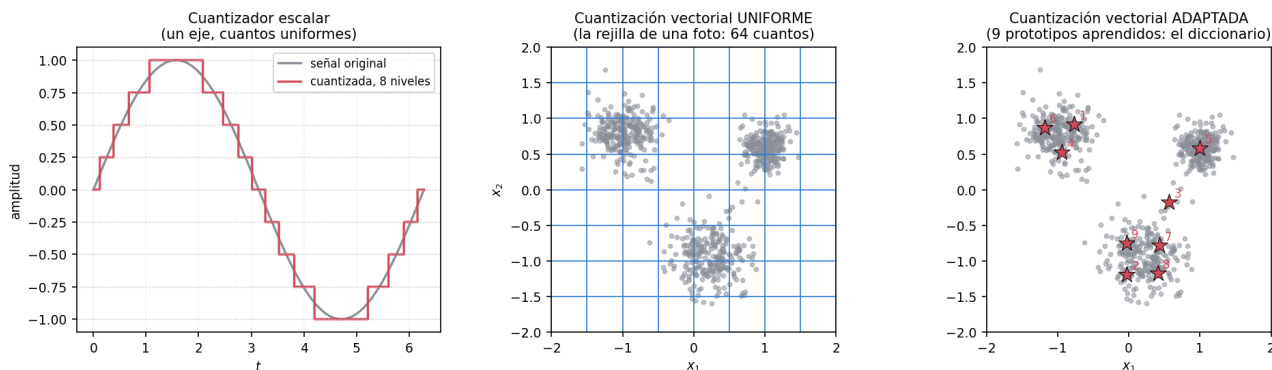


Figura 11: El escalar, el vectorial uniforme y el vectorial adaptado (9 prototipos aprendidos sobre los mismos datos).

Lo interesante es que **no hace falta cuantizar de manera uniforme**. En algunas regiones del espacio hay muchos datos y conviene detalle; en otras no aparece nunca nada y gastar cuantos ahí es tirar bits. De ahí la idea de **cuantización vectorial adaptada a los datos**, que es lo que aprenden estos algoritmos.

Terminología (la misma cosa con otros nombres, para poder leer la bibliografía):

En redes neuronales	En cuantización vectorial
vector de pesos w :	centroide o prototipo m :

Clave	Qué tenés que poder responder
Escalar vs. vectorial	Se cuantiza un eje vs. se cuantiza un espacio
Por qué adaptar	Más detalle donde hay datos, ningún cuanto donde no hay
Diccionario / <i>code-book</i>	El conjunto de prototipos
Para qué sirve	Compresión con pérdida: se guarda el índice del prototipo
Tres formas de entrenarlo	<i>k</i> -medias etiquetado, SOM etiquetado, LVQ

10. El algoritmo LVQ1

1. Inicialización aleatoria. Valores pequeños al azar, o copiando patrones del conjunto de entrenamiento (igual que el SOM). Cada prototipo **tiene asignada una clase desde el principio**: se pone al menos uno por clase, y varios por clase si la distribución es complicada.

2. Selección.

$$c(n) = \arg \min_i \{\|\mathbf{x}(n) - \mathbf{m}_i(n)\|\}$$

3. Adaptación.

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(c, d, n) \alpha [\mathbf{x}(n) - \mathbf{m}_c(n)]$$

$$s(c, d, n) = \begin{cases} +1 & \text{si } \mathcal{C}(c(n)) = d(n) \\ -1 & \text{si } \mathcal{C}(c(n)) \neq d(n) \end{cases}$$

4. Volver a 2 hasta satisfacer el error de clasificación.

OJO — errata de la cátedra (errata 025, 7'58")

La diapositiva escribe la condición como $c(n) = d(n)$ y **está mal**. $c(n)$ es el **índice del prototipo ganador** y $d(n)$ es la **clase del patrón**: son cosas de distinto tipo, no se pueden comparar ($\mathbf{m}_c$ es un vector de $\mathbb{R}^N$, como los datos). Lo que tiene que coincidir con $d(n)$ es $\mathcal{C}(c(n))$, la **clase del prototipo ganador**. En este apunte ya está escrito corregido.

La regla dice: **si el prototipo ganador es de la clase correcta, acercalo al patrón; si no, alejalo.**

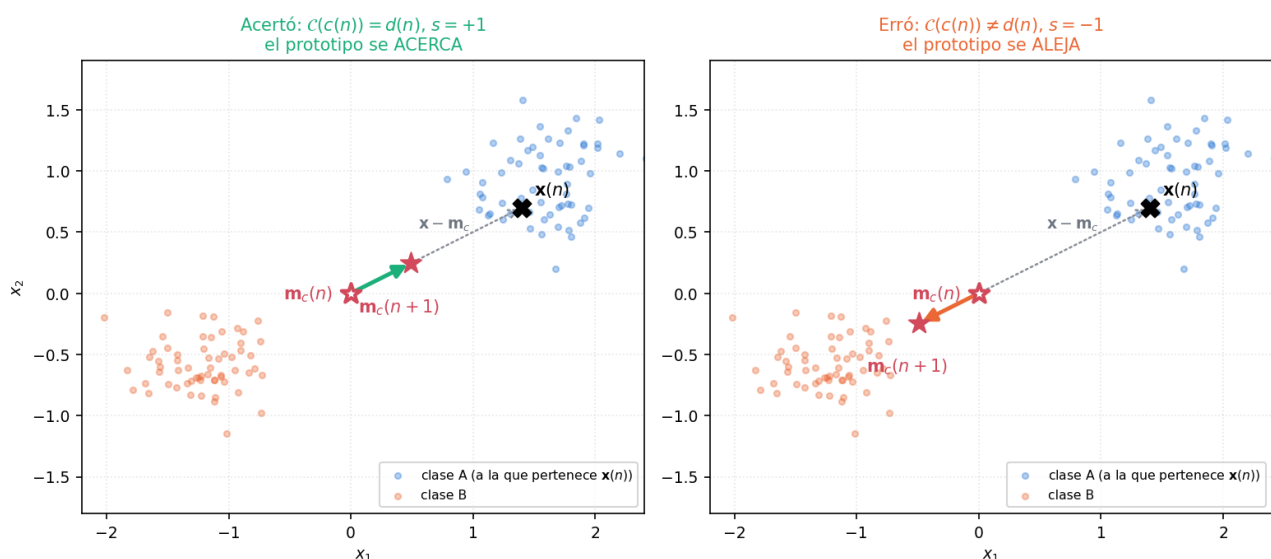


Figura 12: Los dos casos de la diapositiva 47, que en la lámina son dos viñetas sin figura.

Observaciones de la cátedra (diapositivas 47 a 50):

- **No hay arquitectura neuronal.** No hay neuronas ni conexiones: hay prototipos sueltos en el espacio, cada uno puede ir a donde quiera. Es la diferencia central con el SOM.
- Se puede ver al cuantizador como **un SOM lineal, sin entorno y supervisado.**
- Quedan planteadas dos preguntas sobre la velocidad de aprendizaje: ¿existe un α_c óptimo **para cada centroide**? ¿y un $\alpha_c(n)$ óptimo **para cada instante**? (Sección 12.)

Verificación: LVQ1 corriendo de verdad

Dos clases, una de ellas partida en dos nubes separadas (para que un solo prototipo por clase no alcance), 2 prototipos por clase, $\alpha_0 = 0,30$ decayendo un 8 % por época, 40 épocas.

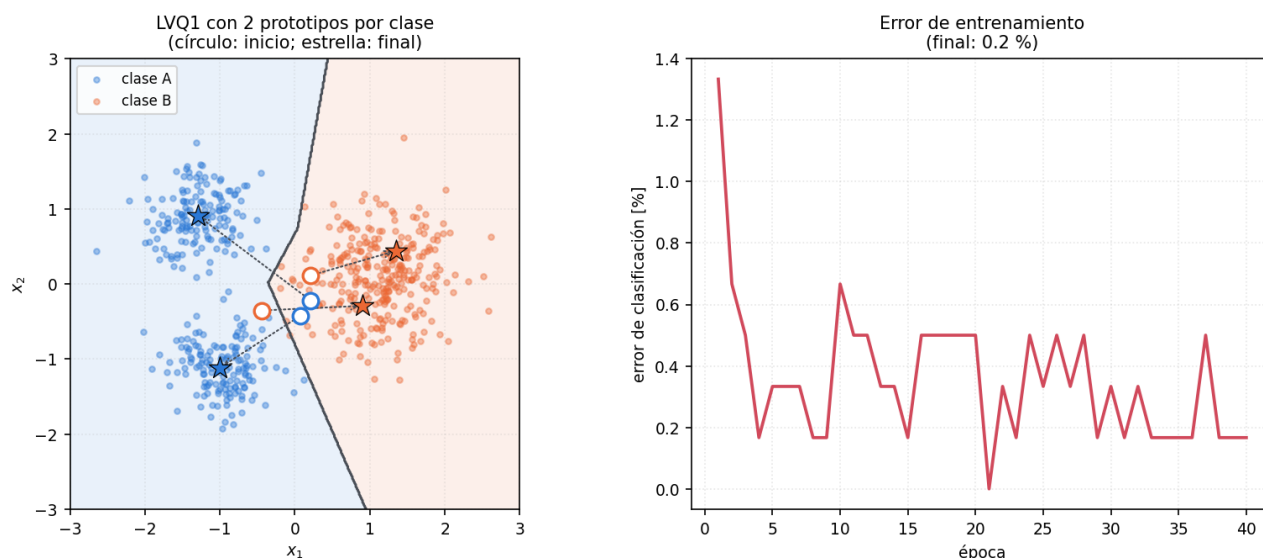


Figura 13: Corrida real: 600 patrones, 4 prototipos, 40 épocas.

El error de clasificación sobre el conjunto de entrenamiento baja de **1,33 %** a **0,17 %**, y los cuatro prototipos terminan uno en cada nube. La frontera que arman es **lineal a trozos**: es la frontera de las regiones de Voronoi de los prototipos, y ahí está el límite del método — con P prototipos no se pueden dibujar fronteras más complicadas que eso.

PARA LA DEFENSA — cuántos prototipos por clase

Es la pregunta natural y la respuesta está en esta figura. Con **un** prototipo por clase, la clase A (dos nubes separadas) habría quedado representada por un punto en el medio de las dos, que no es ninguna de las dos. Con dos por clase, cada nube tiene el suyo. La regla: **tantos prototipos por clase como “pedazos” tenga esa clase en el espacio**, y como eso no se sabe de antemano, se prueba.

Claves de la sección 10

Clave	Qué tenés que poder responder
Los 4 pasos	Inicializar / elegir ganador / acercar o alejar / repetir
$s(c, d, n)$	+1 si acertó la clase, -1 si no
La errata	Va $\mathcal{C}(c(n)) = d(n)$, no $c(n) = d(n)$
Vs. SOM	Sin arquitectura, sin entorno, y supervisado desde el entrenamiento
Frontera que produce	Lineal a trozos: las celdas de Voronoi de los prototipos

11. SOM y LVQ, lado a lado

	SOM	LVQ1
Supervisión	Ninguna durante el entrenamiento	Desde la primera iteración
Arquitectura	Neuronas en un plano, con vecindad	No hay : prototipos sueltos
Entorno Λ_G	Sí, y decreciente	No existe
Quién se actualiza	Ganadora + entorno	Sólo el ganador

	SOM	LVQ1
Sentido del ajuste	Siempre acerca	Acerca si acertó, aleja si erró
Corte	Sin cambios significativos en el mapa	Error de clasificación satisfactorio
Qué produce	Un mapa ordenado topológicamente	Un diccionario de prototipos etiquetados
Se usa para clasificar	Etiquetando después	Directamente

IDEA DE FONDO — una sola ecuación con dos interruptores

$\mathbf{m}_c \leftarrow s \alpha (\mathbf{x} - \mathbf{m}_c)$ es la ecuación del SOM con dos cambios: se le sacó el entorno (sólo se actualiza el ganador) y se le agregó el signo s . Poné $s = +1$ siempre y $\Delta_G = 0$ y tenés k -medias en línea. Agregale entorno y tenés el SOM. Agregale el signo en vez del entorno y tenés LVQ1. Los tres algoritmos de la unidad son la misma línea de código con dos interruptores.

12. LVQ1-O: la velocidad de aprendizaje óptima

Ésta es la parte que la cátedra deja como ejercicio (“Demostrar que...”, diapositiva 62). Va completa.

El problema

Partimos de la regla de adaptación y la desarmamos:

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(n)\alpha(n) [\mathbf{x}(n) - \mathbf{m}_c(n)]$$

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(n)\alpha(n)\mathbf{x}(n) - s(n)\alpha(n)\mathbf{m}_c(n)$$

$$\mathbf{m}_c(n+1) = [1 - s(n)\alpha(n)] \mathbf{m}_c(n) + s(n)\alpha(n) \mathbf{x}(n)$$

Escrita así se lee sola: el prototipo nuevo es una **combinación** del prototipo viejo y del patrón que acaba de entrar, con pesos $[1 - s\alpha]$ y $s\alpha$.

Ahora expandimos un paso más, reemplazando $\mathbf{m}_c(n)$ por lo que valía según la misma ecuación una iteración antes:

$$\mathbf{m}_c(n+1) = [1 - s(n)\alpha(n)] \left\{ \mathbf{m}_c(n-1) + s(n-1)\alpha(n-1) [\mathbf{x}(n-1) - \mathbf{m}_c(n-1)] \right\} + s(n)\alpha(n)\mathbf{x}(n)$$

Y ahí está lo que había que ver: $\mathbf{x}(n-1)$ **queda afectado dos veces por α** — una vez por el $\alpha(n-1)$ con el que entró, y otra por el factor $[1 - s(n)\alpha(n)]$ de la iteración siguiente. $\mathbf{x}(n)$, en cambio, está multiplicado por una sola α .

Si α es constante y menor que 1, esto se repite en cascada: el penúltimo patrón lleva dos factores, el antepenúltimo tres, y así. **La importancia relativa de los primeros patrones de entrenamiento siempre será menor que la de los últimos.** Con $\alpha = 0,5$, un patrón que entró dos iteraciones atrás influye 0,25; diez iteraciones atrás, 10^{-3} . El centroide **se olvida rápido** de lo que se le mostró primero, y su valor final depende casi sólo del final del archivo — que es un orden arbitrario.

IDEA DE FONDO — el problema no es la velocidad, es la equidad

No estamos buscando que converja más rápido. Estamos buscando que el prototipo final sea **el promedio de su grupo** y no un promedio con memoria corta. Que todos los patrones pesen igual es lo que hace que $\mathbf{m}_c$ sea un centroide de verdad.

La condición

Si queremos que α afecte por igual a todos los patrones, tiene que decrecer con el tiempo, y de forma tal que **el peso efectivo del patrón anterior siga siendo el mismo que el del patrón actual**:

$$\underbrace{\alpha_c(n)}_{\text{peso de } x(n)} = \underbrace{[1 - s(n)\alpha_c(n)] \alpha_c(n-1)}_{\text{peso de } x(n-1) \text{ después de este paso}}$$

La demostración (esto es lo que hay que saber hacer en el pizarrón)

Partimos de la condición y despejamos $\alpha_c(n)$:

$$\alpha_c(n) = [1 - s(n)\alpha_c(n)] \alpha_c(n-1)$$

Distribuimos el lado derecho:

$$\alpha_c(n) = \alpha_c(n-1) - s(n) \alpha_c(n) \alpha_c(n-1)$$

Pasamos a la izquierda todo lo que tiene $\alpha_c(n)$:

$$\alpha_c(n) + s(n) \alpha_c(n) \alpha_c(n-1) = \alpha_c(n-1)$$

Sacamos $\alpha_c(n)$ de factor común:

$$\alpha_c(n) [1 + s(n) \alpha_c(n-1)] = \alpha_c(n-1)$$

Y despejamos:

$$\alpha_c(n) = \frac{\alpha_c(n-1)}{1 + s(n) \alpha_c(n-1)}$$

que es lo que había que demostrar. Con la advertencia de la cátedra: **no sobrepasar** $\alpha > 1$.

OJO — de dónde sale la advertencia de $\alpha > 1$

Es el caso $s(n) = -1$ (el prototipo erró). Ahí el denominador es $1 - \alpha_c(n-1)$, que es **menor que 1**, así que la fracción **hace crecer** a α . Si se acumulan errores seguidos, α se dispara por encima de 1 y el prototipo empieza a pasarse de largo del patrón en cada corrección: el algoritmo se vuelve inestable. Por eso se satura. Ésta es la razón concreta de la nota, y es una buena pregunta de parcial.

OJO — hay un α_c por prototipo, no uno solo

El subíndice c no es decorativo: cada prototipo lleva **su propio** α_c , y se actualiza sólo en las iteraciones en las que **ese** prototipo gana. Es la respuesta a la primera de las dos preguntas de la diapositiva 50 (“¿existe un α_c óptimo para cada centroide?”): sí, y es esta recursión.

Verificación numérica

Con $s(n) = +1$ siempre (el caso limpio), la recursión tiene forma cerrada:

$$\alpha_c(n) = \frac{\alpha_0}{1 + \alpha_0(n-1)}$$

que se comprueba por inducción y se verifica en la figura: arrancando en $\alpha_0 = 0,9$ da 0,9, 0,474, 0,321, ..., 0,099 en $n = 10$. α **decae como** $1/n$, que es exactamente el ritmo con el que decae el peso de un dato nuevo en un promedio corrido.

Se calculó el **peso efectivo** de cada patrón $x(k)$ en el prototipo final tras 60 presentaciones, o sea $\alpha(k) \prod_{j>k} [1 - \alpha(j)]$:

	Peso del primer patrón	Peso del último	Relación
$\alpha = 0,30$ constante	$2,2 \times 10^{-10}$	0,300	$1,4 \times 10^9$
$\alpha_c(n)$ óptimo	0,016636	0,016636	1,000000

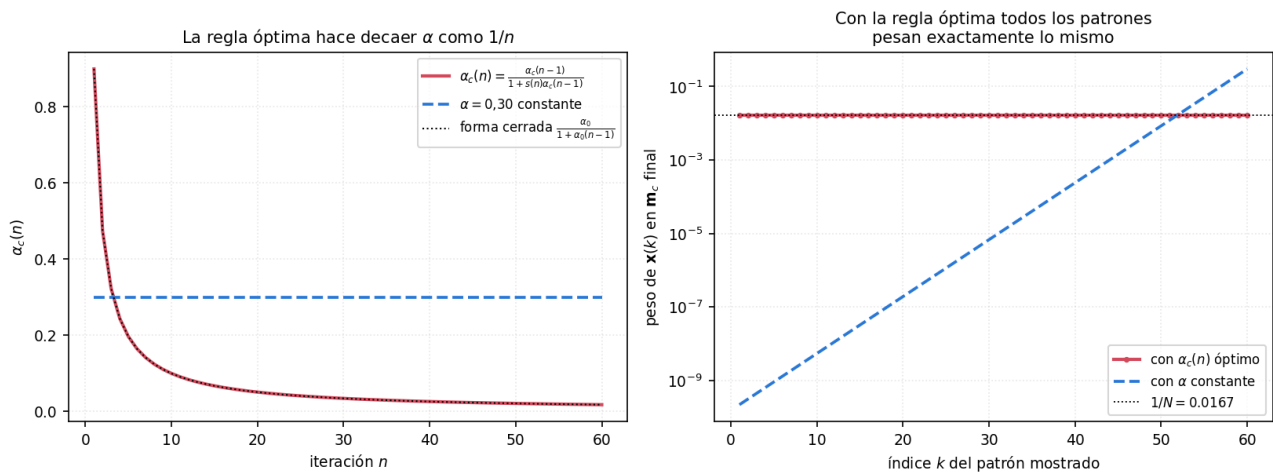


Figura 14: Izquierda: la regla óptima contra un α constante. Derecha: cuánto pesa cada patrón en el valor final del prototipo, en escala logarítmica.

Con α constante el último patrón pesa **mil cuatrocientos millones de veces** más que el primero. Con la regla óptima todos pesan exactamente lo mismo, hasta el último dígito. La demostración funciona.

Claves de la sección 12

Clave	Qué tenés que poder responder
El problema	Con α cte, los primeros patrones pesan mucho menos que los últimos
La forma útil de la regla	$\mathbf{m}_c(n+1) = [1 - s\alpha]\mathbf{m}_c(n) + s\alpha \mathbf{x}(n)$
La condición	$\alpha_c(n) = [1 - s(n)\alpha_c(n)]\alpha_c(n-1)$
El resultado	$\alpha_c(n) = \frac{\alpha_c(n-1)}{1 + s(n)\alpha_c(n-1)}$
Por qué saturar en 1	Con $s = -1$ el denominador es < 1 y α crece
Uno por prototipo	Cada $\mathbf{m}_c$ lleva su α_c , y sólo se actualiza cuando gana

13. El algoritmo completo, paso a paso

La sección 5 tiene el algoritmo tal como lo escribe la cátedra, en cuatro pasos. Ésta es la versión **operativa**: lo mismo, pero con todo lo que hay que decidir para que corra de verdad. Cada paso dice de qué sección sale.

Paso 1 — Definir el mapa

Sección 2 y 3.

Elegir la **forma** (lineal, cuadrada, hexagonal) y el **tamaño** M . Con una cuadrícula de F filas por C columnas, la neurona j tiene dos coordenadas de mapa,

$$\text{fila}(j) = \left\lfloor \frac{j-1}{C} \right\rfloor, \quad \text{columna}(j) = (j-1) \bmod C,$$

que **no se usan nunca para calcular distancias a los patrones**: sirven sólo para el entorno del paso 5.

Paso 2 — Inicialización

Sección 5.

$$w_{ji}(0) \sim \mathcal{U}[-0,5; +0,5] \quad j = 1, \dots, M; \quad i = 1, \dots, N$$

o bien $\mathbf{w}_j(0) = \mathbf{x}_\ell$ con ℓ sorteado. La primera opción arranca con toda la malla apretada cerca del origen — es la maraña del ejemplo 4 — y la segunda ya la reparte sobre los datos.

Se fijan también el cronograma de $\eta(n)$ y $\Lambda_G(n)$ de la sección 6, y $n \leftarrow 0$.

Paso 3 — Tomar un patrón

Sección 5.

Se recorre el archivo de entrenamiento **en orden aleatorio**, un patrón por iteración. Una pasada completa por los L patrones es una **época**; una iteración n es **un patrón**.

Paso 4 — Competencia: la ganadora

Sección 2. $G(\mathbf{x}(n)) = \arg \min_{\forall j} \{\|\mathbf{x}(n) - \mathbf{w}_j(n)\|\}$

Se calculan las M distancias y se toma el índice de la menor. En $\mathbb{R}^N$. Como sólo interesa cuál es la más chica y la raíz es creciente, se puede comparar $\|\mathbf{x} - \mathbf{w}_j\|^2$ y ahorrarse las M raíces: da la misma ganadora.

Paso 5 — Cooperación: el entorno

Sección 3. $\Lambda_G(n) = \{j : \max(|\text{fila}(j) - \text{fila}(G)|, |\text{columna}(j) - \text{columna}(G)|) \leq R(n)\}$

En el mapa. Es la distancia de Chebyshev: un cuadrado, no un círculo. Con $R = 1$ el entorno tiene **9 neuronas si la ganadora está en el interior, 6 si está en un borde y 4 si está en una esquina** — la cuadrícula se recorta, no se envuelve.

Paso 6 — Adaptación

Sección 5.

$$\mathbf{w}_j(n+1) = \mathbf{w}_j(n) + \eta(n) (\mathbf{x}(n) - \mathbf{w}_j(n)) \quad \forall j \in \Lambda_G(n)$$

y las demás **no se tocan**. Con excitación lateral uniforme todas las del entorno reciben lo mismo que la ganadora; si se usara la gaussiana, cada una recibiría $h_{G,j}(n)$ en lugar de $\eta(n)$.

Paso 7 — Avanzar el cronograma

Sección 6.

$n \leftarrow n + 1$. Al terminar cada época se actualizan η y R según la etapa en curso (ordenamiento, transición o ajuste fino). **Dentro de una época se mantienen fijos:** lo que decrece con las épocas, no con cada patrón.

Paso 8 — Corte

Sección 5.

Se vuelve al paso 3 hasta agotar el cronograma o hasta que el mapa no cambie de forma significativa, por ejemplo cuando $\frac{1}{M} \sum_j \|\mathbf{w}_j(n+1) - \mathbf{w}_j(n)\|$ cae por debajo de un umbral.

OJO — el algoritmo termina acá y todavía no clasifica nada

Al salir del paso 8 hay un mapa ordenado y **ninguna etiqueta**. Etiquetar las neuronas es un procedimiento **posterior y supervisado**, y es la sección 8. Confundir las dos cosas es el error típico 3.

Claves de la sección 13

Clave	Qué tenés que poder responder
Coordenadas de mapa	Para qué sirven y para qué no
Paso 4	Se puede comparar la distancia al cuadrado; da la misma ganadora
Paso 5	9 / 6 / 4 neuronas según interior, borde o esquina, con $R = 1$
Paso 7	η y R cambian por época , no por patrón
Después del paso 8	El mapa está ordenado pero sin etiquetas

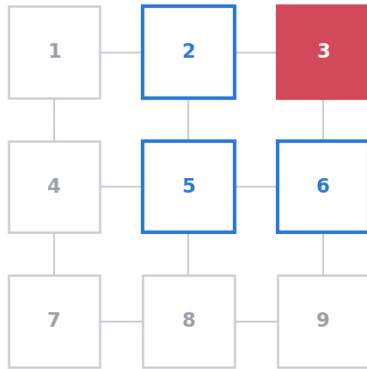
14. Un ejemplo numérico completo, paso por paso

Todo lo anterior sobre un mapa de 3×3 con entradas en $\mathbb{R}^2$, dos iteraciones a mano. Cada bloque dice **qué fórmula aplica y de qué sección sale**. Es el desarrollo que conviene tener practicado: entra en media pizarra y

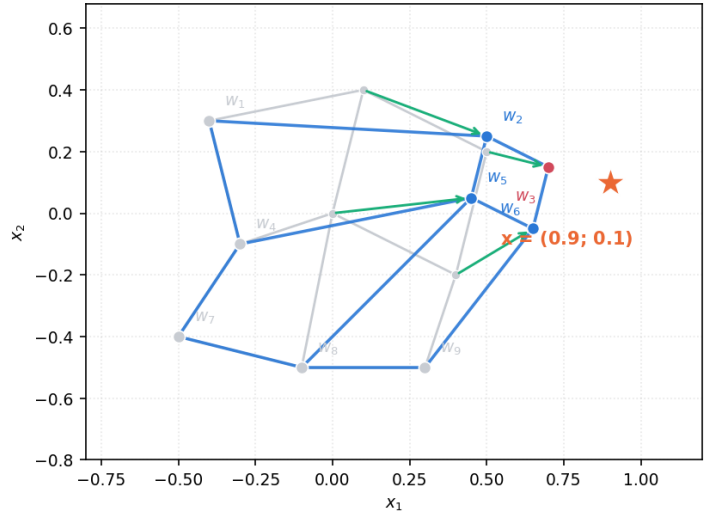
toca las tres piezas del algoritmo.

Dos iteraciones del SOM sobre un mapa de 3×3 con entradas en $\mathbb{R}^2$, $\Lambda_G = 1$.
Gris: posición y malla ANTES de la iteración. Color: DESPUÉS. Rojo: la ganadora.

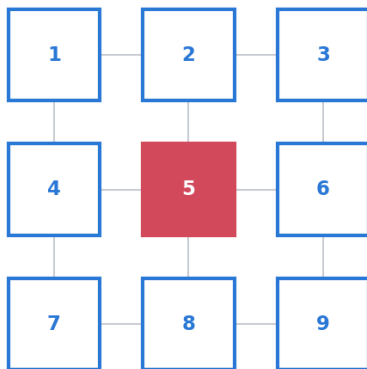
$n = 1$ · en el MAPA: gana la 3; su entorno son 4 neuronas



$n = 1$ · en la ENTRADA: cada peso del entorno camina $\eta = 0.5$ hacia $\mathbf{x}$



$n = 2$ · en el MAPA: gana la 5; su entorno son 9 neuronas



$n = 2$ · en la ENTRADA: cada peso del entorno camina $\eta = 0.5$ hacia $\mathbf{x}$

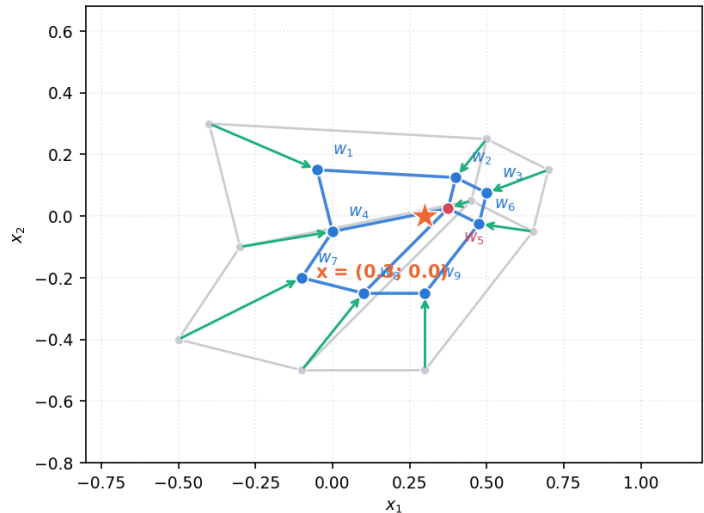


Figura 15: Las dos iteraciones. Izquierda, el mapa: quién gana y a quién arrastra. Derecha, el espacio de entrada: dónde estaban los pesos (gris) y dónde quedaron (color).

Los datos de partida

Mapa de 3×3 , numerado por filas, con $N = 2$ entradas. Los nueve pesos son un sorteo en $[-0,5; 0,5]$ (paso 2 de la sección 13), $\eta = 0,5$ y $\Lambda_G = 1$. El η es deliberadamente grande para que los movimientos se vean; en una corrida real de la etapa 1 andaría por ahí, y en la etapa 3 sería cien veces menor.

$$\begin{array}{lll} \mathbf{w}_1 = (-0,4; 0,3) & \mathbf{w}_2 = (0,1; 0,4) & \mathbf{w}_3 = (0,5; 0,2) \\ \mathbf{w}_4 = (-0,3; -0,1) & \mathbf{w}_5 = (0,0; 0,0) & \mathbf{w}_6 = (0,4; -0,2) \\ \mathbf{w}_7 = (-0,5; -0,4) & \mathbf{w}_8 = (-0,1; -0,5) & \mathbf{w}_9 = (0,3; -0,5) \end{array}$$

Son 18 parámetros: dos por neurona. **No hay umbrales ni sesgos** — el SOM no los tiene, porque el peso no multiplica a nada.

Iteración $n = 1$ — Competencia

Fórmula: §2 y §13 paso 4. $G(\mathbf{x}) = \arg \min_j \|\mathbf{x} - \mathbf{w}_j\|$

Entra $\mathbf{x}(1) = (0,9; 0,1)$. Se calculan las nueve distancias:

j	$\mathbf{x} - \mathbf{w}_j$	$\ \mathbf{x} - \mathbf{w}_j\ ^2$	$\ \mathbf{x} - \mathbf{w}_j\ $
1	(1,3; -0,2)	$1,69 + 0,04 = 1,73$	1,3153
2	(0,8; -0,3)	$0,64 + 0,09 = 0,73$	0,8544
3	(0,4; -0,1)	$0,16 + 0,01 = 0,17$	0,4123
4	(1,2; 0,2)	$1,44 + 0,04 = 1,48$	1,2166
5	(0,9; 0,1)	$0,81 + 0,01 = 0,82$	0,9055
6	(0,5; 0,3)	$0,25 + 0,09 = 0,34$	0,5831
7	(1,4; 0,5)	$1,96 + 0,25 = 2,21$	1,4866
8	(1,0; 0,6)	$1,00 + 0,36 = 1,36$	1,1662
9	(0,6; 0,6)	$0,36 + 0,36 = 0,72$	0,8485

$$G(\mathbf{x}(1)) = 3$$

Fijate que el orden de la columna de los cuadrados es el mismo que el de la columna de las raíces: **por eso se puede comparar sin sacar raíz**.

Iteración $n = 1$ — Cooperación

Fórmula: §3 y §13 paso 5. Distancia de Chebyshev en el mapa, radio 1.

La neurona 3 está en **fila 1, columna 3**: es una **esquina**. Su entorno se recorta contra los dos bordes:

$$\Lambda_G(1) = \{2, 3, 5, 6\} \quad 4 \text{ neuronas, no } 9$$

La 9, por ejemplo, tiene el peso a 0,8485 — más cerca del patrón que la 2, que está a 0,8544 — y **no se actualiza**, porque en el mapa está en la esquina opuesta. La 5, que está a 0,9055, sí. Ése es todo el asunto del SOM en una línea.

Iteración $n = 1$ — Adaptación

Fórmula: §5 y §13 paso 6. $\mathbf{w}_j \leftarrow \mathbf{w}_j + \eta (\mathbf{x} - \mathbf{w}_j)$, con $\eta = 0,5$.

Con $\eta = 0,5$ cada peso queda **en el punto medio** entre donde estaba y el patrón, que es la forma más rápida de controlar la cuenta en el pizarrón:

$$\begin{aligned} \mathbf{w}_2 &= (0,1; 0,4) + 0,5 (0,8; -0,3) = (0,1 + 0,40; 0,4 - 0,15) = (\mathbf{0,500}; \mathbf{0,250}) \\ \mathbf{w}_3 &= (0,5; 0,2) + 0,5 (0,4; -0,1) = (0,5 + 0,20; 0,2 - 0,05) = (\mathbf{0,700}; \mathbf{0,150}) \\ \mathbf{w}_5 &= (0,0; 0,0) + 0,5 (0,9; 0,1) = (0,0 + 0,45; 0,0 + 0,05) = (\mathbf{0,450}; \mathbf{0,050}) \\ \mathbf{w}_6 &= (0,4; -0,2) + 0,5 (0,5; 0,3) = (0,4 + 0,25; -0,2 + 0,15) = (\mathbf{0,650}; \mathbf{-0,050}) \end{aligned}$$

Las otras cinco quedan **exactamente como estaban**: $\mathbf{w}_1, \mathbf{w}_4, \mathbf{w}_7, \mathbf{w}_8, \mathbf{w}_9$ sin tocar. Ésa es la segunda rama de la ecuación.

OJO — la ganadora no es la que más se mueve

La 3 se movió 0,206 y la 5 se movió 0,453: **más del doble**. El paso es proporcional a lo lejos que estaba cada una, y la ganadora es justamente la que estaba más cerca. Lo que gana la ganadora no es un paso más grande, es el derecho a elegir **quiénes** se mueven.

Iteración $n = 2$ — La ganadora queda en el interior

Mismas fórmulas, con los pesos ya actualizados.

Entra $\mathbf{x}(2) = (0,3; 0,0)$:

j	1	2	3	4	5	6	7	8	9
$\ \mathbf{x} - \mathbf{w}_j\ $	0,7616	0,3202	0,4272	0,6083	0,1581	0,3536	0,8944	0,6403	0,5000

$$G(\mathbf{x}(2)) = 5$$

La 5 está en **fila 2, columna 2**: el interior. Ahora el entorno **no se recorta**:

$$\Lambda_G(2) = \{1, 2, 3, 4, 5, 6, 7, 8, 9\} \quad \text{las 9: la ganadora y las 8 de alrededor}$$

Se actualiza el mapa entero. Aplicando la regla con $\eta = 0,5$ a las nueve:

$$\begin{aligned} \mathbf{w}_1 &= (-0,050; 0,150) & \mathbf{w}_2 &= (0,400; 0,125) & \mathbf{w}_3 &= (0,500; 0,075) \\ \mathbf{w}_4 &= (0,000; -0,050) & \mathbf{w}_5 &= (0,375; 0,025) & \mathbf{w}_6 &= (0,475; -0,025) \\ \mathbf{w}_7 &= (-0,100; -0,200) & \mathbf{w}_8 &= (0,100; -0,250) & \mathbf{w}_9 &= (0,300; -0,250) \end{aligned}$$

El control

Cuando el entorno abarca **todo** el mapa, la iteración es una **homotecia de centro $\mathbf{x}$ y razón $(1 - \eta)$** : cada peso queda a $(1 - \eta)$ de la distancia a la que estaba, y todas las distancias entre pesos se multiplican por $(1 - \eta)$. Sale de reescribir la regla en forma de combinación convexa, que es el desarrollo D3 de la sección 15:

$$\mathbf{w}_j(n+1) - \mathbf{x} = (1 - \eta) [\mathbf{w}_j(n) - \mathbf{x}]$$

Con $\eta = 0,5$, entonces, **todo se divide exactamente por dos**. Es el control más rápido de la iteración 2:

	Antes de $n = 2$	Después	Cociente
Largo total de la malla	5,4588	2,7294	0,500
Dispersión media respecto del centroide	0,5206	0,2603	0,500

Si te da otra cosa, te equivocaste en alguna resta.

IDEA DE FONDO — acá se ve la contracción de la etapa 1

Esto explica lo que se ve en el segundo cuadro de la figura del despliegue (sección 7): con un entorno grande **casi toda iteración actualiza a casi todo el mapa**, así que casi toda iteración es una contracción hacia el patrón de turno. Por eso la malla primero se apelotona alrededor de la media de los datos y recién se estira cuando el entorno empieza a achicarse. No es un error de implementación: es la etapa 1 haciendo lo que tiene que hacer.

Qué pasa si se lo deja correr

El mismo mapa de 3×3 , ahora entrenado de verdad sobre 1500 puntos uniformes en $[-1, 1]^2$, 300 épocas con las tres etapas, promediando 8 inicializaciones distintas. Se mide la distancia media $\|\mathbf{w}_a - \mathbf{w}_b\|$ entre **vecinas en el mapa** contra la de **un par cualquiera**:

Corrida	Vecinas en el mapa	Un par cualquiera
Con entorno ($R : 1 \rightarrow 0$)	0,68	1,10
Sin entorno ($R = 0$ siempre)	1,11	1,10

Sin entorno, dos neuronas vecinas en la cuadrícula están tan lejos entre sí como dos cualesquiera: la vecindad del mapa **no informa nada** sobre los datos. Es el mismo resultado que la tabla de la sección 7 sobre un mapa de 6×6 , y es el argumento que conviene tener a mano para el ejercicio de comparación con k -medias.

Claves de la sección 14

Clave	Qué tenés que poder responder
Cuántos parámetros	$M \times N$; no hay umbrales
Por qué se compara $\ \cdot\ ^2$	La raíz es creciente: no cambia el arg min
Entorno en la esquina	4 neuronas; la cuadrícula se recorta, no se envuelve
Quién se mueve más	La que estaba más lejos, no la ganadora
El control con $\eta = 0,5$	Entorno total $\Rightarrow$ todo se divide por dos

15. Cuatro desarrollos para el pizarrón

Los cuatro que conviene tener practicados, con los puntos donde uno se traba.

D1 — El algoritmo de entrenamiento del SOM, de memoria

Desarrollo completo: sección 5, “Para la pizarra”. Versión operativa: sección 13.

Llegás a: los cuatro pasos, con la ecuación de adaptación partida en dos casos según el entorno.

Trampa: escribir la adaptación sin el caso “si $y_j \notin \Lambda_G(n)$ ”. El enunciado tiene dos ramas y la segunda dice *no hagas nada*; si la olvidás, el desarrollo queda igual al de k -medias.

Trampa 2: poner $\arg \max$. Es un **mínimo** de distancia.

D2 — Por qué la vecindad produce el ordenamiento topológico

Llegás a: el argumento de las dos neuronas conectadas. Si j y k son vecinas **en el mapa**, cada vez que gana una se actualiza la otra; entonces las dos son traccionadas hacia las mismas regiones de $\mathbb{R}^N$; entonces $\mathbf{w}_j$ y $\mathbf{w}_k$ terminan cerca **en el espacio de entrada**. Cuando el entorno se apaga, cada una se acomoda en su grupo, pero ya dentro de la misma zona.

Checkpoint: decir explícitamente **en qué espacio se mide cada cosa**. Si no lo decís, el argumento no se entiende.

Trampa: afirmar que el entorno mejora el agrupamiento. **No lo mejora:** sin entorno los pesos también caen en los centros de los grupos (ejemplo 3, error 0,042). Lo único que agrega el entorno es el **orden**.

D3 — La forma “combinación convexa” de la regla de adaptación

Se usa como control numérico en la sección 14.

Llegás a: $\mathbf{m}_c(n+1) = [1 - s\alpha] \mathbf{m}_c(n) + s\alpha \mathbf{x}(n)$, distribuyendo y agrupando.

Trampa: perder el signo al agrupar los dos términos en $\mathbf{m}_c(n)$. Sale $\mathbf{m}_c(n)(1 - s\alpha)$, con el 1 del término original.

Para qué sirve: de acá salen las dos cosas de la sección 12 — la expansión que muestra el doble α , y la condición de equidad.

D4 — La velocidad de aprendizaje óptima

Llegás a: $\alpha_c(n) = \frac{\alpha_c(n-1)}{1 + s(n)\alpha_c(n-1)}$, en cuatro renglones: distribuir, pasar, factor común, despejar.

Trampa: arrancar mal la condición. Del lado izquierdo va $\alpha_c(n)$ **solo** (el peso del patrón actual); del derecho, $\alpha_c(n-1)$ **multiplicado por el factor de olvido** $[1 - s(n)\alpha_c(n)]$. Si escribís el factor con $\alpha_c(n-1)$ adentro no cierra.

Checkpoint: verificá con $s = +1$ y $\alpha_0 = 0,9$: tiene que dar $0,9/1,9 = 0,474$.

16. Formulario

SOM — ganadora

$$j^*(n) = G(\mathbf{x}(n)) = \arg \min_{\forall j} \{ \|\mathbf{x}(n) - \mathbf{w}_j(n)\| \}$$

SOM — adaptación

$$\mathbf{w}_j(n+1) = \begin{cases} \mathbf{w}_j(n) + \eta(n) (\mathbf{x}(n) - \mathbf{w}_j(n)) & y_j \in \Lambda_G(n) \\ \mathbf{w}_j(n) & y_j \notin \Lambda_G(n) \end{cases}$$

Excitación lateral uniforme

$$\Lambda_G(n) = 2 \Rightarrow \begin{cases} h_{G,i} = \beta(n) & |G - i| \leq 2 \\ h_{G,i} = 0 & |G - i| > 2 \end{cases}$$

Excitación lateral gaussiana

$$h_{G,i} = \beta(n) e^{-\frac{|G-i|^2}{2\sigma^2(n)}}$$

Las tres etapas

Etapas	Λ_G	η	Épocas
Ordenamiento	$\approx$ medio mapa	0,9 – 0,7	500 – 1000
Transición	$\rightarrow 1$, lineal	$\rightarrow 0,1$, lineal o exp.	≈ 1000
Ajuste fino	0	0,1 – 0,01, cte	≈ 3000

LVQ1 — ganador y adaptación

$$c(n) = \arg \min_i \{ \|\mathbf{x}(n) - \mathbf{m}_i(n)\| \}$$

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(c, d, n) \alpha [\mathbf{x}(n) - \mathbf{m}_c(n)], \quad s = \begin{cases} +1 & \mathcal{C}(c(n)) = d(n) \\ -1 & \mathcal{C}(c(n)) \neq d(n) \end{cases}$$

LVQ1 — forma combinación

$$\mathbf{m}_c(n+1) = [1 - s(n)\alpha(n)] \mathbf{m}_c(n) + s(n)\alpha(n) \mathbf{x}(n)$$

LVQ1-O — velocidad óptima

$$\alpha_c(n) = [1 - s(n)\alpha_c(n)] \alpha_c(n-1) \quad \Rightarrow \quad \alpha_c(n) = \frac{\alpha_c(n-1)}{1 + s(n)\alpha_c(n-1)}$$

17. Errores típicos

1. **Confundir los dos espacios.** La vecindad se mide en el mapa; la distancia que elige la ganadora, en $\mathbb{R}^N$. Es el error número uno de toda la unidad.
2. **Creer que el entorno sirve para agrupar mejor.** Sirve para **ordenar**. Sin entorno el agrupamiento es igual de bueno (y es k -medias).
3. **Decir que el SOM es supervisado porque se puede clasificar con él.** El entrenamiento es no supervisado; el etiquetado viene **después**.
4. **Escribir $c(n) = d(n)$ en LVQ.** Va $\mathcal{C}(c(n)) = d(n)$: la **clase** del prototipo ganador. Está en la errata 025.
5. **Poner entorno en LVQ.** No hay. No hay ni siquiera arquitectura neuronal: son prototipos sueltos.
6. **Olvidar la segunda rama de la adaptación del SOM** (las de afuera del entorno **no se tocan**).
7. **Mezclar épocas con iteraciones** al citar las duraciones de las tres etapas.
8. **Decir que $\Lambda_G = 2$ son 2 neuronas por lado en diagonal también.** Son las 24 de alrededor: distancia de Chebyshev, un cuadrado.
9. **Dejar α constante en LVQ y decir que converge al centroide.** Converge a algo dominado por los últimos patrones: relación $1,4 \times 10^9$ entre el último y el primero.
10. **Recortar mal el entorno en los bordes.** Con $R = 1$ son 9 neuronas en el interior, 6 en un borde y 4 en una esquina. La cuadrícula **no** se envuelve sobre sí misma.
11. **Olvidar el subíndice c en α_c .** Hay uno por prototipo, y sólo avanza cuando ese prototipo gana.

18. Autoevaluación

Si podés responder estas catorce sin mirar, la unidad está.

1. Definí auto-organización y dá el ejemplo biológico del que salen estas redes.
2. Dibujá la arquitectura del SOM. ¿Cuántas capas tiene? ¿Qué es $\mathbf{w}_j$ y en qué espacio vive?
3. ¿Por qué se dice que es una red **competitiva**? Escribí la ecuación de la ganadora.
4. Dibujá una vecindad cuadrada de radio 1 y una de radio 2. ¿Cuántas neuronas hay en cada una?
5. Escribí las tres funciones de excitación lateral. ¿Cuál se usa en la práctica y por qué la otra es la interesante biológicamente?
6. Escribí los cuatro pasos del entrenamiento. ¿Qué pasa con las neuronas que no están en el entorno?
7. Nombrá las tres etapas con sus valores de Λ_G , η y duración. ¿Qué pasa si te salteás la primera?
8. Explicá con dos neuronas por qué la vecindad produce ordenamiento topológico. ¿En qué espacio se mide cada distancia?
9. ¿Cuál es el interés práctico de un SOM, si k -medias ya agrupa? Contá el *phonetic typewriter*.
10. ¿Cómo se usa un SOM para clasificar? ¿En qué paso entran las clases?
11. Escribí LVQ1 completo. ¿Qué significa $s(c, d, n)$ y cuál es la errata de la diapositiva?
12. Demostrá que $\alpha_c(n) = \alpha_c(n-1)/[1 + s(n)\alpha_c(n-1)]$ y explicá por qué no puede pasar de 1.
13. Mapa de 3×3 , $\Lambda_G = 1$: ¿cuántas neuronas se actualizan si gana la del centro? ¿Y si gana una de una esquina? ¿Por qué?
14. Resolvé una iteración completa a mano: nueve pesos, un patrón, $\eta = 0,5$. Ganadora, entorno y pesos nuevos. Controlá el resultado.